Sprint 4 · Reto «Herramientas internas» · en una página

> Versión corta para **trabajar con ella abierta**. La ficha completa (glosario, uso de la IA, errores frecuentes, guía paso a paso) sigue en Reto_Sprint_04.md; la plantilla de código está en `reto_herramientas/`.

El cliente. TurisData calcula cada semana sus cifras de reservas a mano. Quiere un programa que las calcule solo.
Qué entregáis (no cambia): un repositorio Git con `kpis.py`, `test_kpis.py`, `README.md` y un `salida.json` generado con el CSV real. **Al final del sprint se suben a `sprint-04/` del repositorio del equipo: 4 ficheros del equipo (esos cuatro) + 1 autoevaluación y coevaluación por persona** (`autoevaluacion_nombre.md`).
Tiempo en clase: planning 10 min · reto 100 min · entrega 15 min. Lo que no quepa, a casa o a la 6.ª hora.

Tres niveles: el equipo decide hasta dónde llega

Nivel	Qué hay que tener	Nota*
🟢 **Mínimo** (todos los equipos)	`leer_reservas`, `tasa_cancelacion_por_canal` e `ingreso_medio_por_reserva` pasan sus pruebas · `python kpis.py reservas_turisdata.csv salida.json` crea el JSON (la ocupación y `main` con ayuda del docente) · mensaje claro si falta el fichero · README corto · 3 commits	2
🟡 **Estándar**	Todas las pruebas en verde · errores controlados (fichero inexistente, columna que falta, fila defectuosa descartada y contada) · docstrings y PEP 8 · README con la definición de cada KPI y **por qué la ocupación es solo un proxy** · declaración del uso de IA	3
🔴 **Extra**	Pruebas propias además de las dadas · `kpis_pandas.py` con los mismos resultados · argumento `--por-mes` · `@dataclass` para `Reserva`	4

*Orientativo; puntúa el docente con la rúbrica del curso (0–4 en cada criterio). El mínimo es una entrega válida y evaluable; la «Definition of Done» de la ficha completa describe el nivel **estándar**.

Reparto sugerido de los 100 minutos

Min	Qué	Se sabe que está hecho cuando…
10	Copiar la carpeta y ejecutar `python test_kpis.py`	Veis pruebas fallando (es lo normal)
45	Funciones, **una cada vez**: ejecutar, mirar qué prueba falla, corregir	Cada función pasa sus pruebas
15	`exportar_json` y `main`	Existe `salida.json` con los tres KPIs y `calidad`
15	Romperlo a propósito	Los mensajes de error se entienden
15	README y commits	Otra persona podría ejecutarlo con el README

Frases para el README (completad los huecos)

- «`kpis.py` lee `reservas_turisdata.csv` y calcula … KPIs. Se ejecuta con `python kpis.py … salida.json --capacidad …`.»
- «La ocupación es un **proxy**: contamos todas las noches en la fecha de … y suponemos … habitaciones, así que …»
- «Si falta el fichero, el programa muestra «…» y termina con código 1.»
- «Hemos usado IA para …; lo hemos revisado …; no hemos pegado datos personales.»

Antes de entregar

- **Archivos subidos en `sprint-04/`:** `kpis.py` · `test_kpis.py` · `README.md` · `salida.json` · `autoevaluacion_nombre.md` (**uno por persona**) = 4 + N ficheros
- ☐ `python test_kpis.py` termina sin fallos · ☐ `salida.json` creado con el CSV real · ☐ ≥ 3 commits con mensaje claro
- ☐ README con cómo ejecutar, cómo probar y los KPIs · ☐ Autoevaluación y coevaluación: **una por persona**, subida a `sprint-04/`